

除了AVL树，本章将按照如图7.1所示的总体框架，继续介绍平衡二叉搜索树家族中的其它成员。首先，鉴于数据访问的局部性在实际应用中普遍存在，将按照“最常用者优先”的启发策略，引入并实现伸展树。尽管最坏情况下其单次操作需要 $O(n)$ 时间，但分摊而言仍在 $O(\log n)$ 以内。构思巧妙，实现简洁，加上适用广泛，这些特点都使得伸展树具有别样的魅力。

接下来，通过对平衡二叉搜索树的推广，引入平衡多路搜索树，并着重讨论作为其中典型代表的B-树。借助此类结构，可以有效地弥合不同存储级别之间，在访问速度上的巨大差异。

对照4阶B-树，还将引入并实现红黑树。红黑树不仅能保持全树的适度平衡，从而有效地控制单次操作的时间成本，而且可以将每次重平衡过程执行的结构调整，控制在常数次数以内。后者也是该树有别于其它变种的关键特性，它不仅保证了红黑树更高的实际计算效率，更为持久性结构（persistent structure）之类高级数据结构的实现，提供了直接而有效的方法。

最后，将针对平面范围查询应用，介绍基于平面子区域正交划分的kd-树结构。该结构是对四叉树（quadtree）和八叉树（octree）等结构的一般性推广，它也为计算几何类应用问题的求解，提供了一种基本的模式和有效的方法。

§ 8.1 伸展树

与前一章的AVL树一样，伸展树（splay tree）^①也是平衡二叉搜索树的一种形式。相对于前者，后者的实现更为简捷。伸展树无需时刻都严格地保持全树的平衡，但却能够在任何足够长的真实操作序列中，保持分摊意义上的高效率。伸展树也不需要基本的二叉树节点结构，做任何附加的要求或改动，更不需要记录平衡因子或高度之类的额外信息，故适用范围更广。

8.1.1 局部性

信息处理的典型模式是，将所有数据项视作一个集合，并将其组织为某种适宜的数据结构，进而借助操作接口高效访问。本书介绍的搜索树、词典和优先级队列等，都可归于此类。

为考查和评价各操作接口的效率，除了从最坏情况的角度出发，也可假定所有操作彼此独立、次序随机且概率均等，并从平均情况的角度出发。然而，后一尺度所依赖的假定条件，往往并不足以反映真实的情况。实际上，通常在任意数据结构的生命期内，不仅执行不同操作的概率往往极不均衡，而且各操作之间具有极强的相关性，并在整体上多呈现出极强的规律性。其中最为典型的，就是所谓的“数据局部性”（data locality），这包括两个方面的含义：

- 1) 刚刚被访问过的元素，极有可能在不久之后再次被访问到
- 2) 将被访问的下一元素，极有可能就处于不久之前被访问过的某个元素的附近

充分利用好此类特性，即可进一步地提高数据结构和算法的效率。比如习题[3-6]中的自调

^① 由D. D. Sleator和R. E. Tarjan于1985年发明^[41]

整列表，就是通过“即用即前移”的启发式策略，将最为常用的数据项集中于列表的前端，从而使得单次操作的时间成本大大降低。同样地，类似的策略也可应用于二叉搜索树。

就二叉搜索树而言，数据局部性具体表现为：

- 1) 刚刚被访问过的节点，极有可能在不久之后再次被访问到
- 2) 将被访问的下一节点，极有可能就处于不久之前被访问过的某个节点的附近

因此，只需将刚被访问的节点，及时地“转移”至树根（附近），即可加速后续的操作。当然，转移前后的搜索树必须相互等价，故为此仍需借助7.3.4节中等价变换的技巧。

8.1.2 逐层伸展

■ 简易伸展树

一种直接方式是：每访问过一个节点之后，随即反复地以它的父节点为轴，经适当的旋转将其提升一层，直至最终成为树根。以图8.1为例，若深度为3的节点E刚被访问——无论查找或插入，甚至“删除”——都可通过3次旋转，将该树等价变换为以E为根的另一棵二叉搜索树。

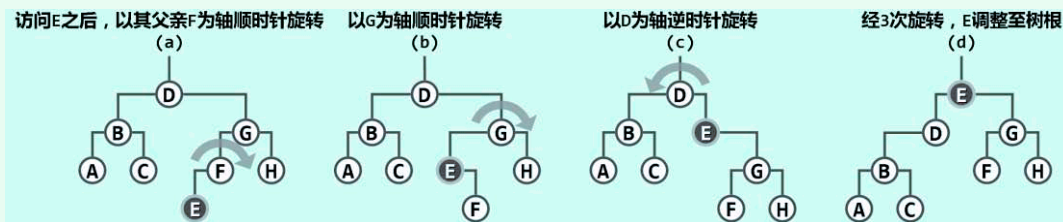


图8.1 通过自下而上的一系列等价变换，可使任一节点上升至树根

随着节点E的逐层上升，两侧子树的结构也不断地调整，故这一过程也称作伸展(splaying)，而采用这一调整策略的二叉搜索树也因此得名。不过，为实现真正意义上的伸展树，还须对以上策略做点微妙而本质的改进。之所以必须改进，是因为目前的策略仍存在致命的缺陷——对于很多访问序列，单次访问的分摊时间复杂度在极端情况下可能高达 $\Omega(n)$ 。

■ 最坏情况

不难验证，若从空树开始依次插入键码{ 1, 2, 3, 4, 5 }，且其间采用如上调整策略，则可得到如图8.2(a)所示的二叉搜索树。接下来，若通过search()接口，再由小到大依次访问各节点一次，则该树在各次访问之后的结构形态将如图(b~f)所示。

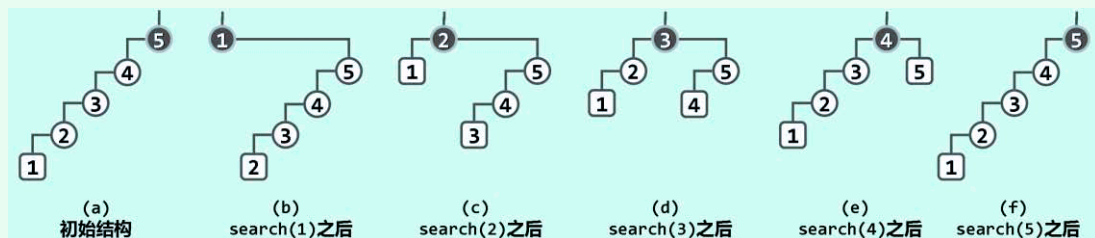


图8.2 简易伸展树的最坏情况

可见，在各次访问之后，为将对应节点伸展调整至树根，分别需做4、4、3、2和1次旋转。

一般地, 若节点总数为 n , 则旋转操作的总次数应为:

$$(n-1) + \{ (n-1) + (n-2) + \dots + 1 \} = (n^2 + n - 2)/2 = \Omega(n^2)$$

如此分摊下来, 每次访问平均需要 $\Omega(n)$ 时间。很遗憾, 这一效率不仅远远低于AVL树, 而且甚至与原始的二叉搜索树的最坏情况相当。而事实上, 问题还远不止于此。

稍做比对即不难发现, 图8.2(a)与(f)中二叉搜索树的结构完全相同。也就是说, 经过以上连续的5次访问之后, 全树的结构将会复原! 这就意味着, 以上情况可以持续地再现。

当然, 这一实例, 完全可以推广至规模任意的二叉搜索树。于是对于规模为任意 n 的伸展树, 只要按关键词单调的次序, 周期性地反复进行查找, 则无论总的访问次数 $m \gg n$ 有多大, 就分摊意义而言, 每次访问都将需要 $\Omega(n)$ 时间!

那么, 这类最坏的访问序列能否回避? 具体地, 又应该如何回避?

8.1.3 双层伸展

为克服上述伸展调整策略的缺陷, 一种简便且有效的方法就是: 将逐层伸展改为双层伸展。具体地, 每次都从当前节点 v 向上追溯两层(而不是仅一层), 并根据其父亲 p 以及祖父 g 的相对位置, 进行相应的旋转。以下分三类情况, 分别介绍具体的处理方法。

■ zig-zig/zag-zag

如图8.3(a)所示, 设 v 是 p 的左孩子, 且 p 也是 g 的左孩子; 设 W 和 X 分别是 v 的左、右子树, Y 和 Z 分别是 p 和 g 的右子树。

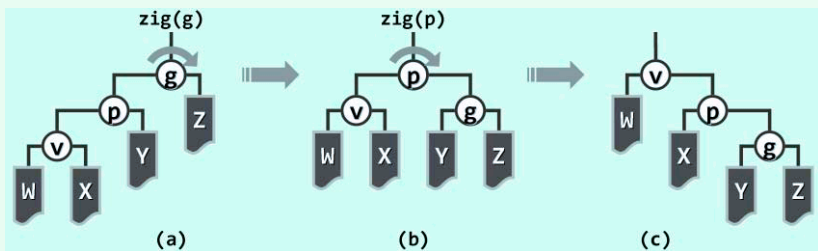


图8.3 通过zig-zig操作, 将节点 v 上推两层

针对这种情况, 首先以节点 g 为轴做顺时针旋转 $\text{zig}(g)$, 其效果如图(b)所示。然后, 再以 p 为轴做顺时针旋转 $\text{zig}(p)$, 其效果如图(c)所示。如此连续的两次 zig 旋转, 合称 zig-zig 调整。

自然地, 另一完全对称的情形—— v 是 p 的右孩子, 且 p 也是 g 的右孩子——则可通过连续的两次逆时针旋转实现调整, 合称 zag-zag 操作。这一操作的具体过程, 请读者独立绘出。

■ zig-zag/zag-zig

如图8.4(a)所示, 设 v 是 p 的左孩子, 而 p 是 g 的右孩子; 设 W 是 g 的左子树, X 和 Y 分别是 v 的左、右子树, Z 是 p 的右子树。

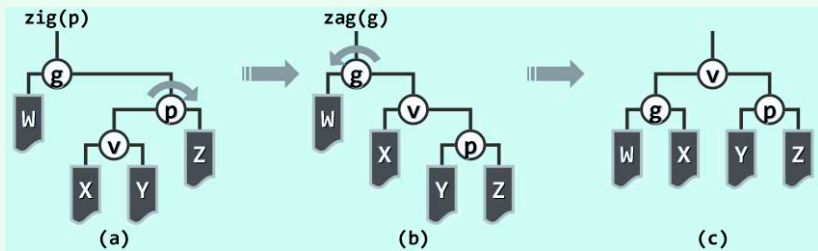


图8.4 通过zig-zag操作, 将节点 v 上推两层

针对这种情况, 首先以节点 p 为轴做顺时针旋转 $\text{zig}(p)$, 其效果如图(b)所示。然后, 再以 g 为轴做逆时针旋转 $\text{zag}(g)$, 其效果如图(c)所示。如此 zig 旋转再加 zag 旋转, 合称 zig-zag 调整。

同样地, 另一完全对称的情形—— v 是 p 的右孩子, 而 p 是 g 的左孩子——则可通过 zag 旋转再加 zig 旋转实现调整, 合称 zag-zig 操作。这一操作的具体过程, 请读者独立绘出。

■ zig/zag

如图8.5(a)所示, 若 v 最初的深度为奇数, 则经过若干次双层调整至最后一次调整时, v 的父亲 p 即是树根 r 。将 v 的左、右子树记作 X 和 Y , 节点 $p = r$ 的另一子树记作 Z 。

此时, 只需围绕 $p = r$ 做顺时针旋转 $\text{zig}(p)$, 即可如图(b)所示, 使 v 最终攀升至树根, 从而结束整个伸展调整的过程。

zag 调整与之对称, 其过程请读者独立绘出。

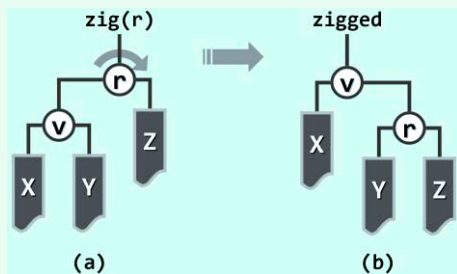


图8.5 通过zig操作, 将节点 v 上推一层, 成为树根

■ 效果与效率

综合以上各种情况, 每经过一次双层调整操作, 节点 v 都会上升两层。若 v 的初始深度 $\text{depth}(v)$ 为偶数, 则最终 v 将上升至树根。若 $\text{depth}(v)$ 为奇数, 则当 v 上升至深度为1时, 不妨最后再相应地做一次zig或zag单旋操作。无论如何, 经过 $\text{depth}(v)$ 次旋转后, v 最终总能成为树根。

重新审视图8.2的最坏实例不难发现, 这一访问序列导致 $\Omega(n)$ 平均单次访问时间的原因, 可以解释为: 在这一可持续重复的过程中, 二叉搜索树的高度始终不小于 $\lfloor n/2 \rfloor$; 而且, 至少有一半的节点在接受访问时, 不仅没有如最初设想的那样靠近树根, 而且反过来恰恰处于最底层。从树高的角度看, 问题根源也可再进一步地解释为: 在持续访问的过程中, 树高依算术级数逐步从 $n - 1$ 递减至 $\lfloor n/2 \rfloor$, 然后再逐步递增回到 $n - 1$ 。那么, 采用上述双层伸展的策略将每一刚被访问过的节点推至树根, 可否避免如图8.2所示的最坏情况呢?

稍作对比不难看出, 就调整之后的局部结构而言, zig-zag和zag-zig调整与此前的逐层伸展完全一致 (亦等效于AVL树的双旋调整), 而zig-zig和zag-zag调整则有所不同。事实上, 后者才是双层伸展策略优于逐层伸展策略的关键所在。以如图8.6(b)所示的二叉搜索树为例, 在 $\text{find}(1)$ 操作之后采用逐层调整策略与双层调整策略的效果, 分别如图(a)和图(c)所示。

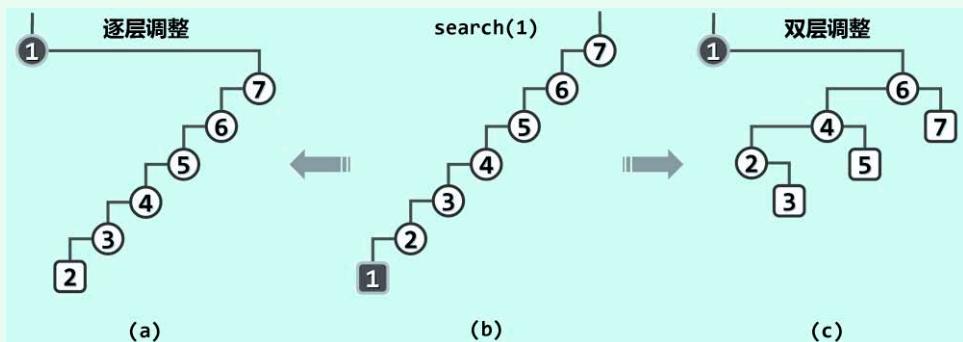


图8.6 双层调整策略的高度折半效果

可见, 最深节点(1)被访问之后再经过双层调整, 不仅同样可将该节点伸展至树根, 而且同时可使树的高度接近于减半。就树的形态而言, 双层伸展策略可“智能”地“折叠”被访问的子树分支, 从而有效地避免对长分支的连续访问。这就意味着, 即便节点 v 的深度为 $\Omega(n)$, 双层伸展策略既可将 v 推至树根, 亦可令对应分支的长度以几何级数 (大致折半) 的速度收缩。

图8.7则给出了一个节点更多、更具一般性的例子，从中可更加清晰地看出这一效果。

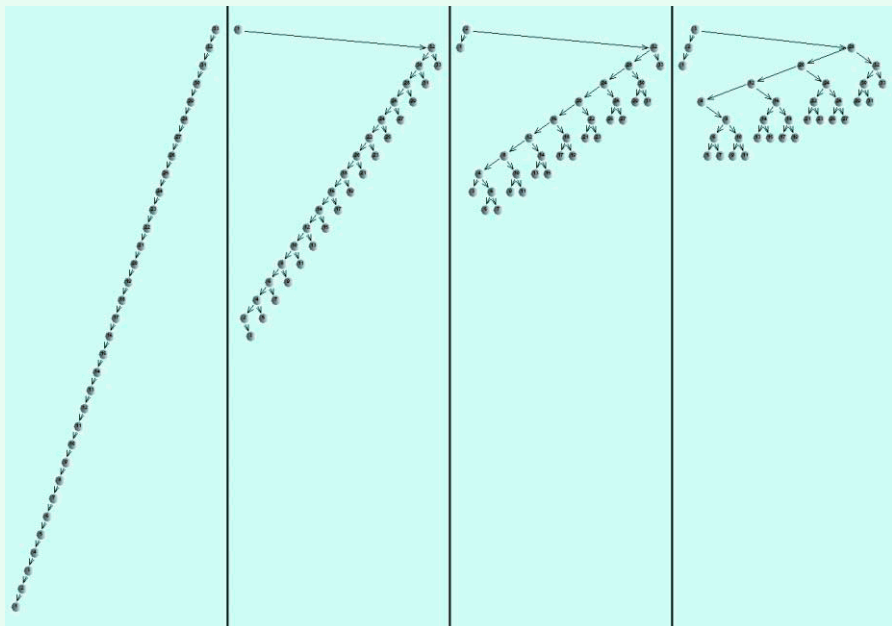


图8.7 伸展树中较深的节点一旦被访问到，对应分支的长度将随即减半

尽管在任一时刻伸展树中都可能存在很深的节点，但与含羞草类似地，一旦这类“坏”节点被“碰触”到，经过随后的双层伸展，其对应的分支都会收缩至长度大致折半。于是，即便每次都“恶意地”试图访问最底层节点，最坏情况也不会持续发生。可见，伸展树虽不能杜绝最坏情况的发生，却能有效地控制最坏情况发生的频度，从而在分摊意义下保证整体的高效率。

更准确地，Tarjan等人^[41]采用势能分析法（potential analysis）业已证明，在改用“双层伸展”策略之后，伸展树的单次操作均可在分摊的 $O(\log n)$ 时间内完成（习题[8-2]）。

8.1.4 伸展树的实现

■ 伸展树接口定义

基于BST类，可定义伸展树模板类Splay如代码8.1所示。

```
1 #include "../BST/BST.h" //基于BST实现Splay
2 template <typename T> class Splay : public BST<T> { //由BST派生的Splay树模板类
3 protected:
4     BinNodePosi(T) splay(BinNodePosi(T) v); //将节点v伸展至根
5 public:
6     BinNodePosi(T) & search(const T& e); //查找（重写）
7     BinNodePosi(T) insert(const T& e); //插入（重写）
8     bool remove(const T& e); //删除（重写）
9 };
```

代码8.1 基于BST定义的伸展树接口

可见，这里直接沿用二叉搜索树类，并根据伸展树的平衡规则，重写了三个基本操作接口

`search()`、`insert()`和`remove()`，另外，针对伸展调整操作，设有一个内部保护型接口`splay()`。

这些接口的具体实现将在以下数节陆续给出。需强调的是，与一般的二叉搜索树不同，伸展树的查找也会引起整树的结构调整，故`search()`操作也需重写。

■ 伸展算法的实现

8.1.3节所述的伸展调整方法，可具体实现如代码8.2所示。

```

1  template <typename NodePosi> inline //在节点*p与*lc (可能为空)之间建立父(左)子关系
2  void attachAsLChild(NodePosi p, NodePosi lc) { p->lChild = lc; if (lc) lc->parent = p; }
3
4  template <typename NodePosi> inline //在节点*p与*rc (可能为空)之间建立父(右)子关系
5  void attachAsRChild(NodePosi p, NodePosi rc) { p->rChild = rc; if (rc) rc->parent = p; }
6
7  template <typename T> //Splay树伸展算法：从节点v出发逐层伸展
8  BinNodePosi(T) Splay<T>::splay(BinNodePosi(T) v) { //v为因最近访问而需伸展的节点位置
9      if (!v) return NULL;
10     BinNodePosi(T) p; BinNodePosi(T) g; //v的父亲与祖父
11     while ((p = v->parent) && (g = p->parent)) { //自下而上，反复对*v做双层伸展
12         BinNodePosi(T) gg = g->parent; //每轮之后*v都以原曾祖父(great-grand parent)为父
13         if (IsLChild(*v))
14             if (IsLChild(*p)) { //zig-zig
15                 attachAsLChild(g, p->rChild); attachAsLChild(p, v->rChild);
16                 attachAsRChild(p, g); attachAsRChild(v, p);
17             } else { //zig-zag
18                 attachAsLChild(p, v->rChild); attachAsRChild(g, v->lChild);
19                 attachAsLChild(v, g); attachAsRChild(v, p);
20             }
21         else if (IsRChild(*p)) { //zag-zag
22             attachAsRChild(g, p->lChild); attachAsRChild(p, v->lChild);
23             attachAsLChild(p, g); attachAsLChild(v, p);
24         } else { //zag-zig
25             attachAsRChild(p, v->lChild); attachAsLChild(g, v->rChild);
26             attachAsRChild(v, g); attachAsLChild(v, p);
27         }
28         if (!gg) v->parent = NULL; //若*v原先的曾祖父*gg不存在，则*v现在应为树根
29         else //否则，*gg此后应该以*v作为左或右孩子
30             (g == gg->lChild) ? attachAsLChild(gg, v) : attachAsRChild(gg, v);
31         updateHeight(g); updateHeight(p); updateHeight(v);
32     } //双层伸展结束时，必有g == NULL，但p可能非空
33     if (p = v->parent) { //若p果真非空，则额外再做一次单旋
34         if (IsLChild(*v)) { attachAsLChild(p, v->rChild); attachAsRChild(v, p); }
35         else { attachAsRChild(p, v->lChild); attachAsLChild(v, p); }
36         updateHeight(p); updateHeight(v);
37     }

```

```

38     v->parent = NULL; return v;
39 } //调整之后新树根应为被伸展的节点，故返回该节点的位置以便上层函数更新树根

```

代码8.2 伸展树节点的调整

■ 查找算法的实现

在伸展树中查找任一关键码 e 的过程，可实现如代码8.3所示。

```

1 template <typename T> BinNodePosi(T) & Splay<T>::search(const T& e) { //在伸展树中查找关键码e
2     BinNodePosi(T) p = searchIn(_root, e, _hot = NULL);
3     _root = splay((p ? p : _hot)); //将最后一个被访问的节点伸展至根
4     return _root;
5 } //与其它BST不同，无论查找成功与否，_root都指向最后被访问的节点

```

代码8.3 伸展树节点的查找

首先，调用二叉搜索树的通用算法`searchIn()`（代码7.3）尝试查找具有关键码 e 的节点。无论查找是否成功，都继而调用`splay()`算法，将查找终止位置处的节点伸展到树根。

■ 插入算法的实现

为将节点插至伸展树中，固然可以调用二叉搜索树的标准插入算法`BST::insert()`（188页代码7.5），再通过双层伸展，将新插入的节点提升至树根。

然而，以上接口`Splay::search()`已集成了`splay()`伸展功能，故查找返回后，树根节点要么等于查找目标（查找成功），要么就是`_hot`，而且恰为拟插入节点的直接前驱或直接后继（查找失败）。因此，不妨改用如下方法实现`Splay::insert()`接口。

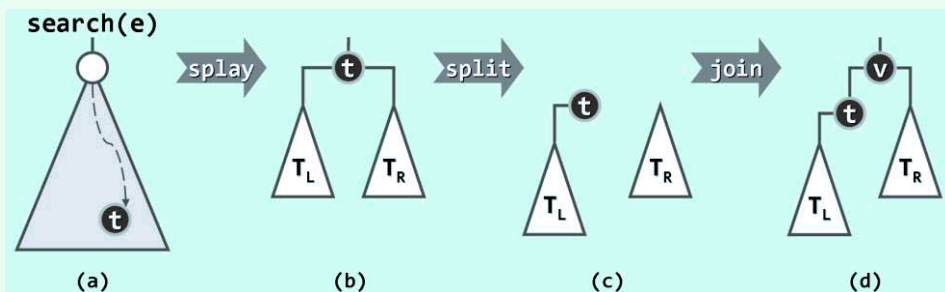


图8.8 伸展树的节点插入

如图8.8所示，为将关键码 e 插至伸展树 T 中，首先调用伸展树查找接口`Splay::search(e)`，查找该关键码（图(a)）。于是，其中最后被访问的节点 t ，将通过伸展被提升为树根，其左、右子树分别记作 T_L 和 T_R （图(b)）。

接下来，根据 e 与 t 的大小关系（不妨排除二者相等的情况），以 t 为界将 T 分裂为两棵子树。比如，不失一般性地设 e 大于 t 。于是，可切断 t 与其右孩子之间的联系（图(c)），再将 e 作为关键码的新节点 v 作为树根，并以 t 作为其左孩子，以 T_R 作为其右子树（图(d)）。

v 小于 t 的情况与此完全对称，请读者独立做出分析。

上述算法过程，可具体实现如代码8.4所示。

```

1 template <typename T> BinNodePosi(T) Splay<T>::insert(const T& e) { //将关键码e插入伸展树中
2     if (!_root) { _size++; return _root = new BinNode<T>(e); } //处理原树为空的退化情况
3     if (e == search(e)->data) return _root; //确认目标节点不存在
4     _size++; BinNodePosi(T) t = _root; //创建新节点。以下调整<=7个指针以完成局部重构
5     if (_root->data < e) { //插入新根,以t和t->rChild为左、右孩子
6         t->parent = _root = new BinNode<T>(e, NULL, t, t->rChild); //2 + 3个
7         if (HasRChild(*t)) { t->rChild->parent = _root; t->rChild = NULL; } //<= 2个
8     } else { //插入新根,以t->lChild和t为左、右孩子
9         t->parent = _root = new BinNode<T>(e, NULL, t->lChild, t); //2 + 3个
10        if (HasLChild(*t)) { t->lChild->parent = _root; t->lChild = NULL; } //<= 2个
11    }
12    updateHeightAbove(t); //更新t及其祖先(实际上只有_root一个)的高度
13    return _root; //新节点必然置于树根,返回之
14 } //无论e是否存在于原树中,返回时总有_root->data == e

```

代码8.4 伸展树节点的插入

尽管伸展树并不需要记录和维护节点高度,为与其它平衡二叉搜索树的实现保持统一,这里还是对节点的高度做了及时的更新。出于效率的考虑,实际应用中可视情况,省略这类更新。

■ 删除算法的实现

为从伸展树中删除节点,固然也可以调用二叉搜索树标准的节点删除算法BST::remove() (190页代码7.6),再通过双层伸展,将该节点此前的父节点提升至树根。

然而,同样鉴于Splay::search()已集成了splay()伸展功能,且在成功返回后,树根节点恰好就是待删除节点。因此,亦不妨改用如下方法实现Splay::remove()接口。

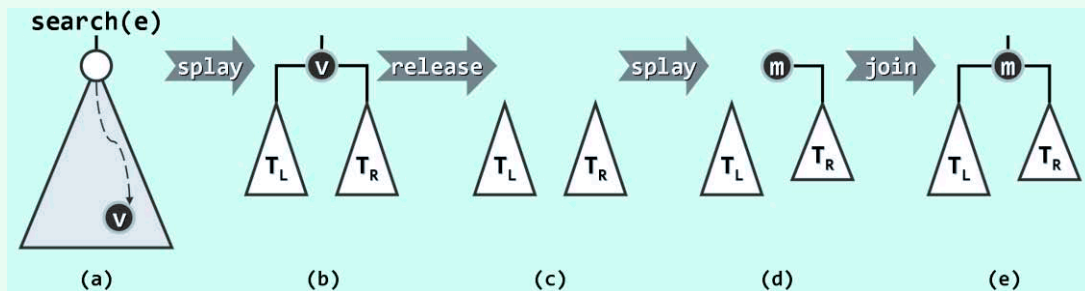


图8.9 伸展树的节点删除

如图8.9所示,为从伸展树T中删除关键码为e的节点,首先亦调用接口Splay::search(e),查找该关键码,且不妨设命中节点为v(图(a))。于是,v将随即通过伸展被提升为树根,其左、右子树分别记作 T_L 和 T_R (图(b))。接下来,将v摘除(图(c))。然后,在 T_R 中再次查找关键码e。尽管这一查找注定失败,却可以将 T_R 中的最小节点m,伸展提升为该子树的根。

得益于二叉搜索树的顺序性,此时节点m的左子树必然为空;同时, T_L 中所有节点都应小于m(图(d))。于是,只需将 T_L 作为左子树与m相互联接,即可得到一棵完整的二叉搜索树(图(e))。如此不仅删除了v,而且既然新树根m在原树中是v的直接后继,故数据局部性也得到了利用。

上述算法过程，可具体实现如代码8.5所示。

```

1 template <typename T> bool Splay<T>::remove(const T& e) { //从伸展树中删除关键码e
2     if (!_root || (e != search(e)->data)) return false; //若树空或目标关键码不存在，则无法删除
3     BinNodePosi(T) w = _root; //assert: 经search()后节点e已被伸展至树根
4     if (!HasLChild(*_root)) { //若无左子树，则直接删除
5         _root = _root->rChild; if (_root) _root->parent = NULL;
6     } else if (!HasRChild(*_root)) { //若无右子树，也直接删除
7         _root = _root->lChild; if (_root) _root->parent = NULL;
8     } else { //若左右子树同时存在，则
9         BinNodePosi(T) lTree = _root->lChild;
10        lTree->parent = NULL; _root->lChild = NULL; //暂时将左子树切除
11        _root = _root->rChild; _root->parent = NULL; //只保留右子树
12        search(w->data); //以原树根为目标，做一次（必定失败的）查找
13        //assert: 至此，右子树中最小节点必伸展至根，且（因无雷同节点）其左子树必空，于是
14        _root->lChild = lTree; lTree->parent = _root; //只需将原左子树接回原位即可
15    }
16    release(w->data); release(w); _size--; //释放节点，更新规模
17    if (_root) updateHeight(_root); //此后，若树非空，则树根的高度需要更新
18    return true; //返回成功标志
19 } //若目标节点存在且被删除，返回true；否则返回false

```

代码8.5 伸展树节点的删除

当然，其中的第二次查找也可在 T_L （若非空）中进行。读者不妨独立实现这一对称的版本。

§ 8.2 B-树

8.2.1 多路平衡查找

■ 分级存储

现代电子计算机发展速度空前。就计算能力而言，ENIAC^②每秒只能够执行5000次加法运算，而今天的超级计算机每秒已经能够执行 3×10^{16} 次以上的浮点运算^③。就存储能力而言，情况似乎也是如此：ENIAC只有一万八千个电子管，而如今容量以TB计的硬盘也不过数百元，内存的常规容量也已达GB量级。

然而从实际应用的需求来看，问题规模的膨胀却远远快于存储能力的增长。以数据库为例，在20世纪80年代初，典型数据库的规模为10~100 MB，而三十年后的今天，典型数据库的规模已需要以TB为单位来计量。计算机存储能力提高速度相对滞后，是长期存在的现象，而且随着时间的推移，这一矛盾将日益凸显。鉴于在同等成本下，存储器的容量越大（小）则访问速度越慢（快），因此一味地提高存储器容量，亦非解决这一矛盾的良策。

^② 第一台电子计算机，1946年2月15日诞生于美国宾夕法尼亚大学工学院

^③ 2013年6月，天河-2以此运算速度，荣登世界超级计算机500强榜首